STW4003CEM
Object Oriented Programming

Week 1

Overview

Topics Covered:

- Overview of Programming
- Introduction to Java
- Variables and Data Types
- Operators, Loops, Conditional Statements
- Methods and OOP in Java
- Four Pillars of OOP (Encapsulation, Inheritance, Abstraction, Polymorphism)
- Exception Handling, Data Structures, Java Swing

Assessment Details

Module Credits: 15 assessment credits & 20 learning credits

CW1 (67%) – Test 1:

- 1-hour online in-class test
- Focus: Programming (MCQs & Coding)

CW2 (33%) – Test 2:

- 1-hour online in-class test
- Focus: Theoretical knowledge (Coding, MCQs & Subjective Questions)

What is a Computer Program?

A computer program is a sequence of instructions written using a Computer Programming Language to perform a specified task by the computer.

Examples: Word processors, web browsers, video games, databases.

Introduction to Programming

Programming is the process of writing and executing instructions for a computer.

Two key steps:

- Writing Source Code in a programming language.
- Converting Source Code into Machine Code.

Types of Programming Languages

Low-Level Languages

- Machine Language (Binary code: 0s & 1s)
- Assembly Language (Mnemonics, requires an assembler)

High-Level Languages

- Procedural → C, Pascal
- Scripting → JavaScript, PHP
- Functional → Haskell, Scala
- Object-Oriented → Java, Python, C#

Compiled vs. Interpreted Languages

Compiled Languages: Convert entire source code into machine code before execution (e.g., C, C++, Java).

Interpreted Languages: Execute code line by line (e.g., Python, JavaScript, PHP).

Introduction to Java

General-purpose, object-oriented, and network-centric programming language.

Originally Called: Oak, developed for portable devices.

Rebranded to Java: In 1995, to capitalize on web development.

Acquisition by Oracle: 2009, including Java, MySQL, and Solaris.

Features of Java

Easy-use programming language

Code once run it on almost any computing platform

Platform-independent

Designed to build object-oriented applications

Multithreaded language with automatic memory management

Created for the distributed environment of the Internet.

Facilitates distributed computing as its network-centric

Java Platform

A collection of programs that help developers to run Java applications.

Works with various operating systems.

Components:

- Execution Engine: Runs Java programs.
- Compiler: Converts Java code into bytecode.
- Libraries: Provides pre-built code for various functionalities.

Components of Java

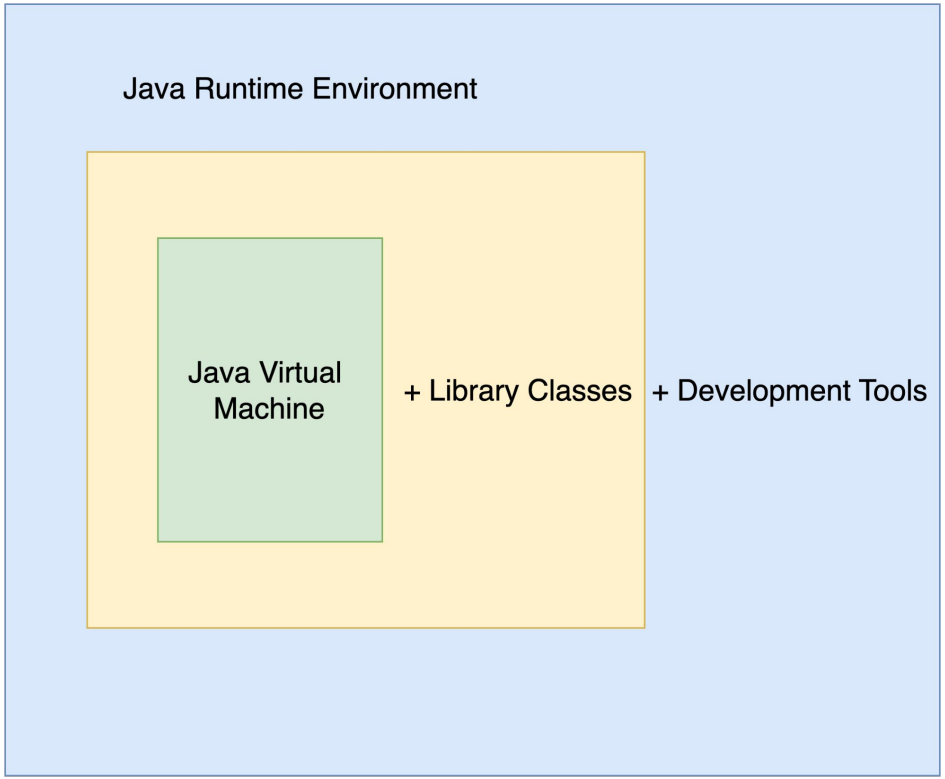
Java Language: Object-oriented programming language used to write Java applications.

JDK (Java Development Kit): A set of development tools for creating Java applications.

JRE (Java Runtime Environment): A set of tools that enable running Java applications.

Java Compiler: Converts Java code into platform-independent bytecode.

JVM (Java Virtual Machine): Executes bytecode, making Java platform-independent.



JDK = Java Runtime Environment + Development Tools
JRE = Java Virtual Machine + Library Classes

Java Compiler

It converts .java files into .class files.

The generated .class files contain Java bytecode, which can be executed on any platform that has a JVM installed.

Just-in-Time (JIT) Compilation:

The JVM uses a JIT compiler to convert bytecode into machine-specific code just before execution.

This improves performance by compiling code at runtime.

Java Virtual Machine

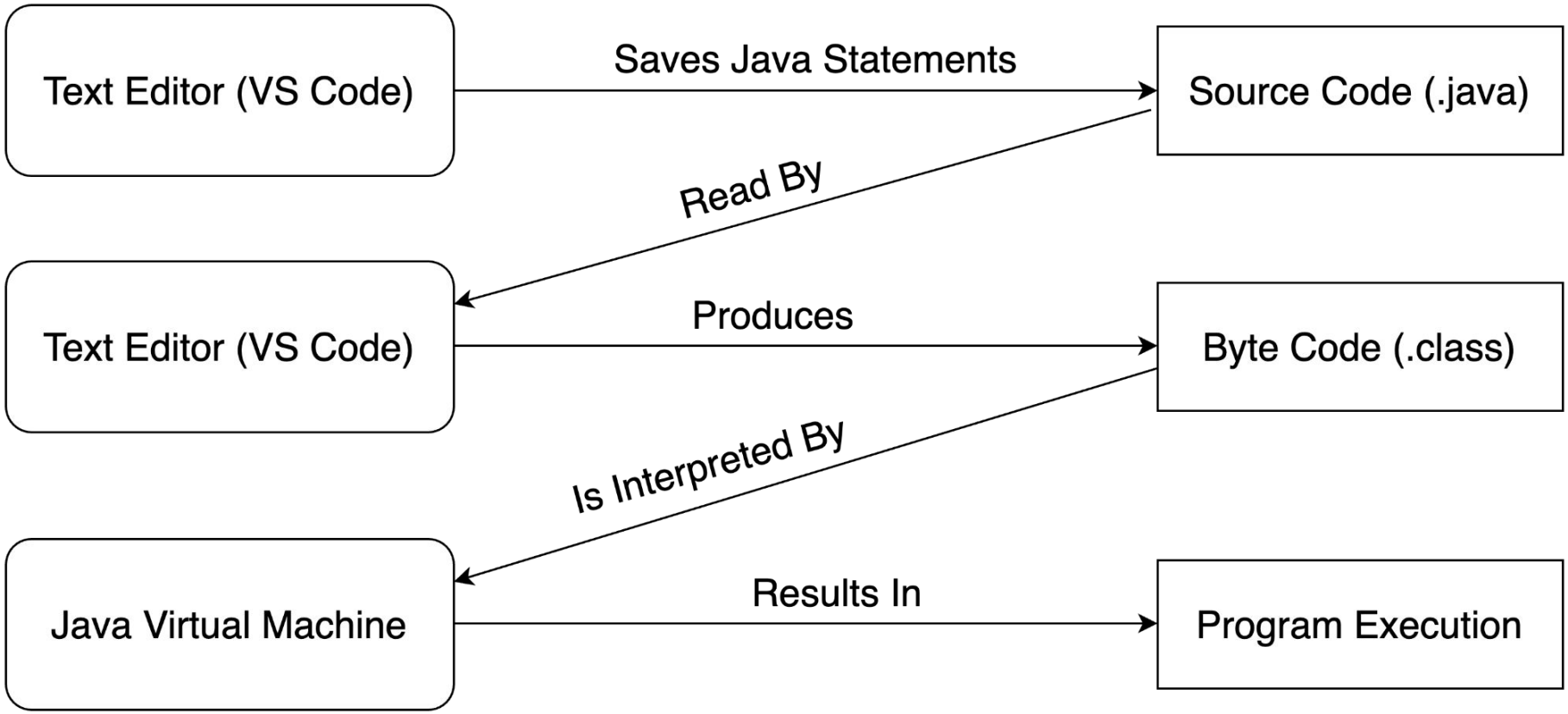
The core of the Java platform, responsible for executing Java bytecode.

Functionality:

- Converts Java bytecode into machine-specific code (also called object code).
- Manages memory and handles garbage collection (removes unused memory).
- Ensures security when running Java applications.

JVM works on various operating systems, making Java programs portable.

JVM's behavior can be adjusted using a virtual interface, independent of the machine or OS.


Text Editor (VS Code)
Saves Java Statements
Source Code (.java)
Read By
Text Editor (VS Code)
Produces
Byte Code (.class)
Is Interpreted By
Java Virtual Machine
Results In
Program Execution

Java Installation

Download via link: https://code.visualstudio.com/docs/java/java-tutorial

Or,

Go to extension and Install : Project Manager for Java and Extension Pack for Java (JDK)

Java Program Structure

Main Components:

- Class: The building block of Java programs.
- Method: Contains code that gets executed.
- Statements: Commands to be executed, ending with semicolons.

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Naming Conventions in Java

Classes: Use PascalCase (e.g., HelloWorld).

Methods: Use camelCase (e.g., displayMain()).

Variables: Use camelCase (e.g., userName).

Java Keywords

Reserved words in Java that cannot be used as identifiers.

Examples:

public, class, static, if, else, try, catch, etc.

Escape Sequences in Java

Allows insertion of special characters into strings.

Examples:

- \n - New line
- \" - Double quotation mark
- \\ - Backslash
- \t - Tab (indentation)

Java Development Process

Steps:

- Write the Code: In a .java file.
- Compile: Using the Java compiler (javac) to produce .class files.
- Run: Use the Java Virtual Machine (JVM) to execute the bytecode.

Process Flow:
Java source code → Compiler → Bytecode → JVM → Machine Code

Week 2

Data Types in Java

Define the type of data a variable can store (e.g., integers, decimals, characters). They help the operating system allocate the correct amount of memory for each variable.
Types of Data Types:

- Primitive Data Type: Predefined data types in Java.
- Non-Primitive Data Type: User-defined types in Java.

Primitive Data Types

Predefined data types in Java. They are used to store simple values.

Examples include:

int: Stores integers (e.g., int i = 100;)

float: Stores floating-point numbers (e.g., float f = 1.23f;)

boolean: Stores true or false (e.g., boolean b = true;)

char: Stores single characters (e.g., char c = 'A';)

Non-Primitive Data Types

User-defined types in Java. They can store references to objects or other types of data.

String: Stores sequences of characters (e.g., "Hello").

Array: Stores multiple variables of the same type.

Class: User-defined types to create objects.

Interface: Declares methods but without implementations.

Primitive vs Non-Primitive Data Types

Feature	Primitive Data Types	Non-Primitive Data Types
Definition	Predefined in Java	Defined by programmer
Example	int , float, boolean , char	String , Array , Class , Interface
Can be null	No	Yes
Memory size	Fixed	Variable
Storing value	Stores value directly	Stores reference to value

Variables in Java

A variable is a named memory location used to store data during a program's execution. Its value can change, hence the name "variable."

Declaration: A variable must be declared by specifying its name and type.

Example:

- int age;
- int age = 25;

Types of Variables

Local Variable: Declared inside methods, constructors, or blocks. It is created when the block is entered and destroyed when it is exited.

Instance Variable: Declared inside a class but outside methods. Each object has its own copy of instance variables.

Static Variable: Declared using the static keyword. Only one copy exists for the entire class, shared among all objects of that class.

Java Type Casting

Widening Casting (Implicit Casting): Automatically converts a smaller type to a larger type.

Example: int -> long, float -> double.

Narrowing Casting (Explicit Casting): Manually converts a larger type to a smaller type.

Example: (int) 5.7 converts 5.7 to 5.

Operators in Java

Arithmetic Operators: Used for basic math operations. Example: +, -, *, /, %

Assignment Operators: Used to assign values to variables. Example: =, +=, -=

Relational Operators: Used to compare values. Example: ==, !=, <, >

Logical Operators: Used to perform logical operations. Example: &&, ||, !

Unary Operators: Operate on a single operand. Example: ++, --, +, -

Ternary Operator

Arithmetic Operators

Perform basic mathematical operations.

Operators: +, -, *, /, %.

Assignment Operators

Used to assign values to variables.

Operators: =, +=, -=, *=, /=.

Relational Operators

Used to compare two values.

Operators: ==, !=, >, <, >=, <=.

Logical Operators

Used for logical operations, usually with boolean values.

Operators: &&, ||, !.

Unary Operators

Used with a single operand to perform operations like increment or decrement.

Operators: ++, --, +, -, !.

Ternary Operator

A shorthand for if-else statements.

Syntax: (condition) ? expression1 : expression2;

User Output in Java

Java provides multiple methods to display output:

- System.out.print() - Prints text without moving to the next line.
- System.out.println() - Prints text and moves to the next line.
- System.out.printf() - Prints formatted text (similar to C/C++).

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
        System.out.print("Hello, ");
        System.out.printf("Hello, %s!", "World");
    }
}
```

Output

```
Hello, World!
Hello, Hello, World!
```

User Input in Java

Scanner Class is commonly used for input in Java. It is available in the java.util package.

Steps to use:

- Import Scanner: import java.util.Scanner;
- Create Scanner object: Scanner scan = new Scanner(System.in);
- Use methods like nextLine(), nextInt(), etc., to read input.

```java
import java.util.Scanner;

public class ScannerExample {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = scan.nextLine();
        System.out.println("Your name is "+ name);

    }
}
```

Output

```
Enter your name: Sangya
Your name is Sangya
```

User Input in Java Using args

Command-line arguments allow users to pass parameters to a program when it’s executed from the terminal or command line.

How it works:

- Command-line arguments are passed as an array of strings to the main() method in Java.
- The args parameter is an array of String objects.

Accessing Arguments:

- args[0] refers to the first argument passed.
- args[1] refers to the second argument, and so on.

```java
public class CommandLineExample {
    public static void main(String[] args) {
        // Check if enough arguments are provided
        if (args.length >= 2) {
            System.out.println("First Argument: " + args[0]);
            System.out.println("Second Argument: " + args[1]);
        } else {
            System.out.println(x:"Not enough arguments provided.");
        }
    }
}
```

```
sangyakoirala@Sangyas-MacBook-Air-2 test % javac CommandLineExample.java
sangyakoirala@Sangyas-MacBook-Air-2 test % java CommandLineExample Hello World
First Argument: Hello
Second Argument: World
```

Parsing Data in Java

Java provides methods like parseInt(), parseDouble(), etc., to convert a string into its respective data type.

Common Parsing Methods:

- Integer.parseInt(String s)
- Double.parseDouble(String s)
- Boolean.parseBoolean(String s)

Task To Do

Week 3

Java Control Statements

Java executes code from top to bottom. The flow of the execution can be altered by some statements.

1. Decision Making statements

- if statements
- switch statement

2. Loop statements

- for loop
- while loop
- do while loop
- for-each loop

3. Jump statements

- break statement
- continue statement

Decision-Making Statements

Control the flow of execution in a program based on conditions.

if-else Statement

- if statement, if-else, if-else-if

Switch Case

- Switch-case for multiple conditions
- Nested switch-case

if-else Statement

Used when you want to check a condition and perform actions based on whether the condition is true or false.

If Block: Executes a block of code if the condition is true.

if-else Statement: Executes one block of code if the condition is true, another if false.

```
public class IfElseExample {
    public static void main(String[] args) {
        int a= 5;
    //   if block example
        if (a>2){
            System.out.println(x:"The number is greater than 2");
        }
    //  if else statement example
        if (a>2){
            System.out.println(x:"The number is greater than 2");
        } else {
            System.out.println(x:"The number is smaller than or equal to 2");
        }
    }
}
```

if-else-if Ladder

Multiple conditions tested in sequence.

```
public class IfElseExample {
    public static void main(String[] args) {
       int a= 5;
        if (a>2){
            System.out.println(x:"The number is greater than 2");
        } else if (a>1){
            System.out.println(x:"The number is greater than 2 and smaller than or equal to 2");
        } else {
            System.out.println(x:"The number is smaller than or equal to 2");
        }
    }
}
```

Nested if

An if statement inside another if statement.

```
public class NestedIf {
    public static void main(String[] args) {
        int a = 10, b = 5, c = 15;

        if (a > b) {
            if (a > c) {
                System.out.println(x:"a is the largest");
            } else {
                System.out.println(x:"c is the largest");
            }
        } else {
            System.out.println(x:"b is the largest");
        }
    }
}
```

Switch Case

Used to select one of many code blocks to be executed based on the value of a variable.

```
public class IfElseExample {
    public static void main(String[] args) {
       int a= 5;
        switch (a){
            case 1:
                System.out.println(x:"The number is 1");
                break;
            case 2:
                System.out.println(x:"The number is 2");
                break;
            case 3:
                System.out.println(x:"The number is 3");
                break;
            default:
                System.out.println(x:"The number is not 1, 2 or 3");
        }

    }
}
```

Nested Switch Case Statement

Switch case inside switch case

```
public class NestedSwitch {
    public static void main(String[] args) {
        int day = 2, time = 1;

        switch (day) {
            case 1: System.out.println(x:"Monday"); break;
            case 2:
                System.out.println(x:"Tuesday");
                switch (time) {
                    case 1: System.out.println(x:"Morning"); break;
                    case 2: System.out.println(x:"Evening"); break;
                }
                break;
        }
    }
}
```

Use of If Statements and Switch Case Statements

Criteria	If Statements	Switch Case Statements
Number of Conditions	Use when there are fewer conditions (typically < 4-5)	Use when there are multiple conditions (typically > 4-5)
Condition Type	Use for boolean expressions (e.g., comparisons)	Use for comparing a single variable to multiple constant values
Complex Conditions	Use for complex conditions (e.g., combining multiple variables or logical operators)	Not suitable for complex boolean conditions, best for simpler checks
Different Actions for Conditions	Use when you need different blocks of code for each condition	Suitable when actions for each condition are clear and involve discrete values
Readability	Less readable with many conditions, especially nested	More readable with many conditions involving the same variable
Handling Discrete Values/Enums	Not ideal for many fixed values, leads to nested if-else	Ideal for checking a variable against multiple discrete values (e.g., days of the week, integers)
Best Use Case	Flexible, complex conditions or boolean expressions	Fixed value checks for cleaner, more readable code

Week 4

Iteration

Iteration refers to the repetition of a process until a condition is met.

In programming, iterations are performed using loops.

Loops help execute code multiple times efficiently.

Java provides three types of loops: For loop, While loop, Do-While loop.

Use of Loops

Automates repetitive tasks.

Reduces redundancy in code.

Improves efficiency and maintainability.

Example: Displaying a message 100 times using a loop instead of writing 100 statements.

Increment & Decrement Operators

Used to update loop variables.

Increment (++) & Decrement (--)

Two types:

Prefix (++var / --var): Changes value before evaluation.

Postfix (var++ / var--): Changes value after evaluation.

Loops in Java

For Loop - Used when the number of iterations is known.

While Loop - Used when the number of iterations is unknown and based on a condition.

Do-While Loop - Executes the loop body at least once, even if the condition is false.

For Loop

```
1  public class Loop {
2      public static void main(String[] args) {
3          for (int i=1; i<=5; i++ ){
4              System.out.println(i);
5          }
6      }
7  }
8
```

Output

```
1
2
3
4
5
```

While Loop

```java
public class Loop {
    public static void main(String[] args) {
        int i = 1;
      while (i<5){
          System.out.println(i);
          i++;
      }
    }
}

```

Output

```
1
2
3
4
```

Do While Loop

```
1 public class Loop {
2     public static void main(String[] args) {
3         int i = 6;
4       do {
5           System.out.println(i);
6           i++;
7       }while (i<5);
8     }
9 }
```

Output

```
6
```

When to Use Loop

Loop	Condition
For Loop	When the number of iterations is known.
While Loop	When iterations depend on a condition.
Do-While Loop	When the loop must run at least once.

Jump Statements

Transfer the execution control to the other part of the program.

break: used to break current execution flow of the program. We can use break statement inside loop, switch case etc.

continue: used to skip the current iteration of the loop.

Tasks To Do

Week 5

What are Java Methods?

A method is a block of code that runs only when called.

It improves code reusability and readability.

Java has two types of methods:

User-Defined Methods – Created by programmers.

Standard Library Methods – Predefined in Java libraries.

Declaring and Calling Methods

A method in Java is like a set of instructions that runs only when called.

A method has three parts:

- Return Type – What type of value it gives back (e.g., int, void).
- Method Name – The name used to call the method.
- Parameters – The values it takes (optional).

Method Return Types & Parameters

Some methods return values, while others don’t.

If a method doesn’t return anything, use void.

```
public class Methods {
   static int addNumbers(int a, int b) {
   return a + b;
}
    public static void main(String[] args) {
        int sum = addNumbers(5, 10); // Calls the method and stores result
        System.out.println(sum);
}
}
```

Standard Library Methods & Advantages

Java already provides some methods that can be used directly.

Examples:

- System.out.println("Hello") → Prints text.
- Math.sqrt(25) → Finds square root (output: 5.0).

Advantages of Using Methods

- Code Reusability – Write once, use multiple times.
- Better Readability – Code is easier to understand.
- Easy Debugging – Fixing errors is simpler.

Tasks To Do

Week 6

Arrays in Java

Arrays store multiple values in a single variable where elements are of the same data type and stored in contiguous memory.
Arrays are index-based (starting from 0).

Creating Array

- Method 1: Declaration then allocation → int[] arr; arr = new int[5];
- Method 2: Declaration & allocation → int[] arr = new int[5];
- Method 3: Declaration, allocation & initialization → int[] arr = {100, 98, 60};

Accessing and Modifying Arrays

- Use index to access values: marks[0] = 100;
- Array length: marks.length

Types of Arrays

Single-Dimensional Array → int[] arr = new int[5];

Multi-Dimensional Array → int[][] arr = new int[3][3];

```java
public class Arrays {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40};
        int sum = 0;
        for (int num : numbers) {
            sum += num;
        }
        System.out.println("Sum: " + sum);
    }
}
```

```java
public class Arrays {
    public static void main(String[] args) {
        int[][] matrix = {{1, 2}, {3, 4}};
        System.out.println(matrix[0][1]);
    }
}
```

Advantages & Disadvantages

Advantages:

- Code Optimization
- Random Access

Disadvantages:

- Fixed size (Cannot grow dynamically)

ArrayList in Java

A part of the Java Collection Framework

Present in java.util package

Implements the List interface

Allows duplicate elements

Maintains insertion order

Allows random access (index-based)

Features of ArrayList

Can contain duplicate elements

Provides methods for add, remove, get, set

Maintains insertion order

Supports random access (index-based retrieval)

Slower than LinkedList for modifications (due to shifting)

Cannot store primitive data types (use wrapper classes)

Why ArrayList?

Limitations of Arrays:

- Fixed size
- Cannot add/remove elements dynamically
- Requires creating a new array for modifications

ArrayList as a Solution:

- Uses a dynamic array
- No size limit, elements can be added/removed anytime
- More flexible than arrays

```java
import java.util.ArrayList;
public class ArrayLists {
    public static void main(String[] args) {
        // declaring ArrayList
        ArrayList<String> list = new ArrayList<>();
        // adding values
        list.add(e:"A");
        list.add(e:"B");
        System.out.println(list);
        // modifying element
        list.set(index:1, element:"C");
        System.out.println(list);
        // accessing element
        System.out.println(list.get(index:1));
        // removing element
        list.remove(o:"B");
        System.out.println(list);
    }
}
```

Output

```
[A, B]
[A, C]
C
[A, C]
```

Array vs. ArrayList

Feature	Array	ArrayList
Size	Fixed	Dynamic
Performance	Faster	Slower (due to resizing)
Data Type	Can store primitives	Needs wrapper class
Flexibility	Requires manual resizing	Auto-resizing

Tasks To Do

Week 7

Object Oriented Programming

- Programming using objects
- In real world, everything is an object
- Object has properties(entities) and behaviours(methods)
- Objects need class

Procedural Programming

Focuses on functions and procedures that operate on data.

Limitations

- Changing data structures affects multiple functions.
- Complex function hierarchies make programs:
- Difficult to understand and maintain.
- Hard to modify and extend.
- Prone to errors.

Object Oriented Programming

Organizes code into objects containing data and methods.

Tries to map code to real-world entities.

Key advantages:

Faster and easier execution.

Clear program structure.

DRY (Don’t Repeat Yourself) principle.

Code reusability and modularity.

Object Oriented Programming

First OOP Language: Simula (1960s).

Popular OOP Languages: Java, C++, Python, C#, PHP.

Core OOP Concepts:

- Objects
- Classes
- Inheritance
- Encapsulation
- Abstraction
- Polymorphism

Building Blocks of OOP

Class: Blueprint for objects.

Object: Instance of a class.

Methods: Define object behaviors.

Attributes: Object properties.

Example of Class & Object

Class: Dog

Attributes: Name, Age

Methods: Bark(), Eat()

Object: "Buddy" (Dog, 3 years old)

Four Pillars of OOP

Encapsulation: Hides internal details.

Inheritance: Enables code reuse via parent-child relationships.

Abstraction: Simplifies complex systems by exposing only necessary parts.

Polymorphism: Allows objects to take multiple forms.

Procedural vs Object Oriented Programming

Feature	Procedural Programming	Object Oriented Programming
Focus	Functions and procedures	Objects and methods
Code Reusability	Limited	High (Inheritance, Polymorphism)
Data Security	Low	High (Encapsulation)
Scalability	Difficult	Easier
Example	C, Pascal	Java, Python, C++

Access Modifiers

Access Modifiers control class, method, and variable visibility.

Four types:

- Default (Package-Private)
- Public
- Private
- Protected

Access Modifiers In Java

Access Modifier	Within the Class	Other Classes [Within the Package]	In Subclasses [Within the package and other packages]	Any Class [In Other Packages]
public	Y	Y	Y	Y
protected	Y	Y	Y	N
default	Y	Y	Same Package – Y Other Packages - N	N
private	Y	N	N	N

Y – Accessible
N – Not Accessible

Default Access Modifier

No explicit modifier used.

Visible only within the same package.

```
class DefaultAccess {

    public static void main(String[] args) {
        System.out.println(x:"Default access");
    }
}
```

Public Access Modifier

Accessible from any class in any package.

```
class PublicAccessModifier {
    public void display() {
        System.out.println(x:"Private access");
    }
}

public class Example {
    public static void main(String[] args) {
        PublicAccessModifier obj = new PublicAccessModifier();
        obj.display();
    }
}
```

Private Access Modifier

Accessible only within the same class.

Prevents access from outside classes.

```
class PrivateAccessModifier {
    private int x = 10;
    private int y = 20;
    private void display() {
        System.out.println(x + y);
    }
}
public class Example {
    public static void main(String[] args) {
        PrivateAccessModifier obj = new PrivateAccessModifier();
        // Compile Time Error
        // obj.display();
    }
}
```

Protected Access Modifier

Accessible within the same package and by subclasses.

Useful in Inheritance

```
class Parent {
    protected void show() {
        System.out.println(x:"Protected access");
    }
}

class Child extends Parent {
    void display() {
        show();  // Allowed
    }
}
```

this Keyword

Refers to the current instance of a class.

Used to:

- Refer to instance variables.
- Call class methods or constructors.
- Pass the current instance as an argument.

Use of this Keyword

```
// without 'this' keyword
class ExampleOne {
    int x;
       void setX(int a) {
          x = a;  // 'x' refers to local variable
       }
}
// with 'this' keyword
class ExampleTwo {
    int x;

    void setX(int x) {
        this.x = x; // 'this.x' refers to instance variable
    }
}
```

this Keyword in Constructor

```java
// this keyword in Constructor
public class KeywordUseInConstructor {
    int a;
    int b;

    KeywordUseInConstructor(int a, int b) {
        this.a = a;
        this.b = b;
    }
    void display() {
        System.out.println("a = " + a + " b = " + b);
    }
    public static void main(String[] args) {
        KeywordUseInConstructor obj = new KeywordUseInConstructor(a:10, b:20);
        obj.display();
    }
}
```

Java Constructor

Special method invoked at object creation to initialize objects.

Has same name as the class and no return type.

Only called once at creation of object.

Types:

- Default (No-Arg) Constructor
- Parameterized Constructor
- Copy Constructor

```
class Example {
    Example() {
        // constructor body
    }
}
```

Non-Parameterized Constructor

No parameters are passed.

If not defined, default constructor is created with default values.

```
public class ConstructorExample{
    int a;
    public ConstructorExample(){
        System.out.println(x:"Default Constructor is called");
        a=10;
    }
    public static void main(String[] args){
        ConstructorExample obj = new ConstructorExample();
        System.out.println("Value of a is: "+obj.a);
    }
}
```

Parameterized Constructor

Includes parameters

Used to provide different values to distinct objects.

```
public class ConstructorExample{
    int a;
    // parameterized constructor
    public ConstructorExample(int a){
        System.out.println(x:"Parameterized Constructor is called");
        this.a = a;
    }
    public static void main(String[] args){
        ConstructorExample obj = new ConstructorExample(a:10);
        System.out.println("Value of a: "+obj.a);
    }
}
```

Copy Constructor

Overloaded constructor used to declare and initialize an object from another object

Can only be one user-defined constructor

```
class ConstructorExample {
    int x;
    ConstructorExample(int val) {
        x = val;
    }
    ConstructorExample(ConstructorExample obj) {
        x = obj.x;
    }
    public static void main(String[] args) {
        ConstructorExample obj1 = new ConstructorExample(val:10);
        ConstructorExample obj2 = new ConstructorExample(obj1);
        System.out.println(obj1.x, obj2.x);
    }
}
```

S.N.	When to Use Constructor	When to use Methods
1.	Initializing object state and allocating resources.	Performing operations or calculations on objects.
2.	Setting default values for object properties.	Manipulating and modifying object data.
3.	Performing any necessary setup operations.	Returning results or values based on object state.
S.N.	**Where to Use Constructor**	**Where to Use Methods**
1.	Inside a class definition define how objects of that class should be initialized.	Inside a class definition to define the behavior of objects of that class. In other parts of the code where the object needs to perform specific operations.
2	Constructors focus on initializing objects and setting their initial state.	While methods provide the functionality and behavior of the objects.

Feature	Constructor	Method
Purpose	Initializes the object and sets initial value	Performs operations and manipulations on the objects
Invocation	Automatically invoked when object is created	Invoked explicitly by the code or other methods
Name	Same as the class name	Can have valid identifier name
Multiple Invocation	Can only be invoked once during object creation	Can be invoked multiple times throughout the object's lifecycle
Inheritance	Cannot be inherited but can be called explicitly in derived classes	Can be inherited and overridden in derived classes

Garbage Collection

Java automatically deallocate unused objects.

- Frees memory.
- Prevents memory leaks.

Identifies objects without references and removes them.

Runs automatically but can be requested using: *System.gc()*

JVM decides the best time to run it.

Getters and Setters in Java

Getters and Setters provide controlled access to private fields.

- Getters **(Accessor Methods**): Retrieve the value of private fields
- Setters (**Mutator Methods**): Modify the value of private fields

Ensure encapsulation and controlled access

Tasks To Do

Week 8

Encapsulation

Fundamental concept in Object-Oriented Programming (OOP).

Bundling data (fields) and methods (functions) that operate on that data into a single unit (class).

Prevents unauthorized access and modification of class data.

Features of Encapsulation

Data Binding: Groups data and related methods inside a class.

Data Hiding: Restricts access to certain data fields.

Access Control: Uses private fields and public methods (getters and setters).

Improves Code Readability & Maintainability.

Why Use Encapsulation?

Maintains Code Clarity: Keeps related data and functions together.

Controls Field Values: Allows validation before assigning values.

Decouples System Components: Enables independent development & testing.

Enhances Security: Prevents direct modification of sensitive fields.

Encapsulation Example

```java
public class Person {
    private int age;

    // Getter Method
    public int getAge() {
        return age;
    }

    // Setter Method with Validation
    public void setAge(int newAge) {
        if (newAge > 0) {
            age = newAge;
        }
    }
}
```

Encapsulation and Data Hiding

Encapsulation does not always mean data hiding.

Data can still be accessed if methods allow it.

True data hiding occurs when fields are made private and access is strictly controlled.

Benefits of Encapsulation

Full control over data members and methods.

Prevents direct modification of sensitive data.

Enhances security & code maintainability.

Supports code reusability & modularity.

Facilitates unit testing & debugging.

Modern IDEs provide quick access to getters & setters.

Tasks To Do

Week 9

Inheritance

Process by which one class acquires the properties (methods and fields) of another

Super Class (Parent/ Base Class) : class whose properties are inherited

Sub Class (Derived/Child Class) : class that inherits the properties of another

extends Keyword: used to inherit the Parent Class

```java
public class Parent {
    public void print() {
        System.out.println(x:"Parent");
    }
}

class classA extends Parent {
    public void print() {
        System.out.println(x:"classA");
    }
}
```

```java
//super or parent class
class Employee {
    void salary() {
        System.out.println(x:" Salary = 40000");
    }
}

// sub class or child class
class Programmer extends Employee {
    // Programmer class inherits from Employee class
    void bonus() {
        System.out.println(x:"Bonus = 10000");
    }
}

public class ExampleOfInheritance {
    Run | Debug
    public static void main(String[] args) {
        Programmer p = new Programmer();
        p.salary(); // calls method of super class
        p.bonus(); // calls method of sub class
    }
}
```

Output

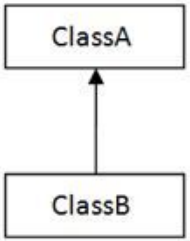
```
 Salary = 40000
Bonus = 10000
```

Key Terms Used in Inheritance

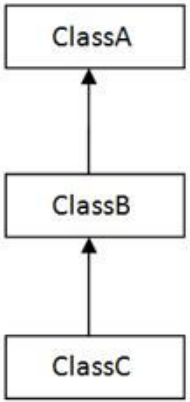
Sub Class/Child Class: A subclass is a class that inherits from another class. It is also called a derived class, an extended class, or a child class.

Super Class/Parent Class: A superclass is the class from which a subclass inherits features. It is also called a base class or a parent class.

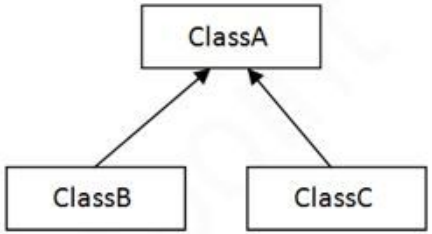
Reusability: It is a mechanism that allows you to reuse the fields and methods of an existing class when you create a new class and add new methods to it as well.



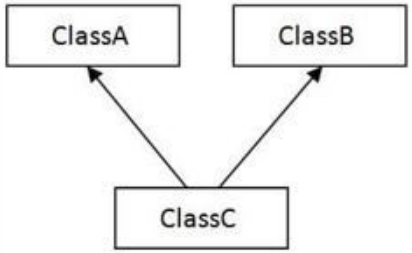
1) Single



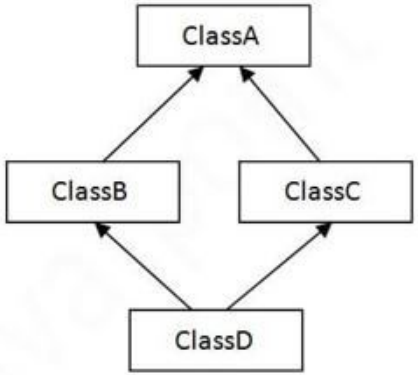
2) Multilevel



3) Hierarchical



4) Multiple



5) Hybrid

In Java programming, multiple and hybrid inheritance is supported through the interface only. We will learn about interfaces later.

Single Inheritance

Sub-class is derived from only one super class

```
class Teacher {
    void teach() {
        System.out.println(x:"Teaching...");
    }
}
class Student extends Teacher {
    void learn() {
        System.out.println(x:"Learning...");
    }
}
public class Example {
    public static void main(String[] args) {
        Student obj = new Student();
        obj.teach();
        obj.learn();
    }
}
```

Multilevel Inheritance

Class is derived from a class which is also derived from another class

```
class GrandParent {
    void grandParentMethod() {
        System.out.println(x:"Grand Parent Method");
    }
}
class Parent extends GrandParent {
    void parentMethod() {
        System.out.println(x:"Parent Method");
    }
}
class Child extends Parent {
    void childMethod() {
        System.out.println(x:"Child Method");
    }
}
```

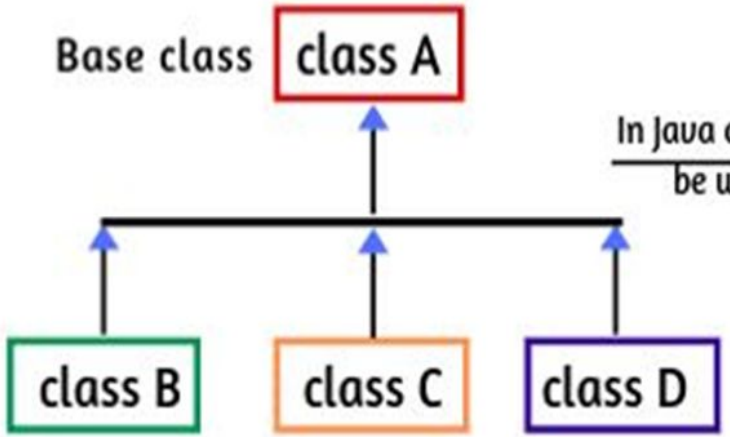
```
public class Example {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Child obj = new Child();
        obj.grandParentMethod();
        obj.parentMethod();
        obj.childMethod();
    }
}
```

Hierarchical Inheritance

Number of classes are derived from a single base class

```
class bird {
    void fly() {
        System.out.println(x:"Flying");
    }
}
class parrot extends bird {
    void talk() {
        System.out.println(x:"Talking");
    }
}
class crow extends bird {
    void walk() {
        System.out.println(x:"Walking");
    }
}
```

```
class Example {
    public static void main(String[] args) {
        parrot p = new parrot();
        p.fly();
        p.talk();
        crow c = new crow();
        c.fly();
        c.walk();
    }
}
```



Here, class A is the parent class (or base class) for all three classes B, C, and D.

In Java code, this can be written as:

```
class  A
 {
    // fields & methods
}
class B extends A {
  // fields & methods
 }
class C extends A  {
  ..... }
class D extends A {
  .....
}
```

Why Use Inheritance?

Code Reusability: Inherits properties and methods from the parent class, reducing code duplication.

Extensibility: Subclasses can add or override methods, extending functionality without modifying the parent.

Maintainability: Centralized updates in the parent class propagate to all subclasses, ensuring consistency.

Polymorphism: Allows objects of subclasses to be treated as objects of the parent class, enabling flexible and reusable code.

Logical Hierarchy: Reflects real-world relationships in a structured way, making the code easier to understand.

Inheritance and Constructors

Constructors are not inherited

When a subclass is instantiated, the superclass default constructor is executed first

When a derived class is extended from the base class, the constructor of the base class is executed automatically, followed by the constructor of the derived class

```java
class BaseClass {
    BaseClass() {
        System.out.println(x:"Base class constructor");
    }
}
// Derived class
class ChildClass extends BaseClass {
    ChildClass() {
        System.out.println(x:"Derived class constructor");
    }
    void greet() {
        System.out.println(x:"Hello");
    }

}
class Example {
    public static void main(String[] args) {
        ChildClass child = new ChildClass();
        child.greet();
    }
}
```

Output

```
Base class constructor
Derived class constructor
Hello
```

Constructor Overloading during Inheritance

Concept of having more than one constructor with different parameters so that every constructor can perform a different task

```java
class Parent {
    Parent() {
        System.out.println(x:"Constructor of Parent");
    }
    Parent(int num) {
        System.out.println(x:"arg constructor of Parent");
    }
}
class Child extends Parent {
    Child() {
        System.out.println(x:"Constructor of Child");
    }
}
public class Example {
    public static void main(String[] args) {
        Child obj = new Child();
    }
}
```

Output

```
Constructor of Parent
Constructor of Child
```

super Keyword

Similar to this keyword

Use Cases:

- Refer to an immediate parent class instance variable.
- Invoke the immediate parent class method.
- Invoke the immediate parent class constructor.

Immediate parent class instance variable

```
class Animal {
    String color = "white";
}
class Dog extends Animal {
    String color = "black";
    void printColor() {
        System.out.println(color); // prints color of Dog class
        System.out.println(super.color); // prints color of Animal class
    }
}
public class Example {
    public static void main(String args[]) {
        Dog d = new Dog();
        d.printColor();
    }
}
```

Output

```
black
white
```

Invoke the immediate parent class method

```java
class Parent {
    void display() {
        System.out.println(x:"Parent class method");
    }
}
class Child extends Parent {
    void display() {
        System.out.println(x:"Child class method");
    }
    void printMsg() {
        display();
        super.display();
    }
}
public class Example {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.printMsg();
    }
}
```

Output

```
Child class method
Parent class method
```

Invoke the immediate parent class constructor

```
class Parent{
    Parent(){
        System.out.println(x:"Parent class constructor");}
}
class Child extends Parent{
    Child(){
        super();
        System.out.println(x:"Child class constructor");}
}
public class Example{
    public static void main(String args[]){
        Child obj = new Child();
    }
}
```

Output

```
Parent class constructor
Child class constructor
```

IS-A Relationship

IS-A is a way of saying, This object is a type of that object

According, to the figure,

- Mammal is an Animal
- Dog is an Animal
- Dog is a Mammal
- Reptile is an Animal

```
class Animal{}
class Mammal extends Animal{}
class Dog extends Mammal{}
class Reptile extends Animal{}
```

Tasks To Do

Week 10

Abstraction

Process of hiding the implementation details and showing only functionality to the user

Shows only essential things to the user and hides the internal details

Focus on what the object does instead of how it does it

Benefits of Abstraction

Simplicity: Simplifies complex code with clear interfaces.

Modularity: Breaks systems into manageable, independent modules.

Encapsulation: Hides internal details, preventing misuse.

Reusability: Promotes reuse of components across projects.

Reduced Complexity: Hides unnecessary details, easing understanding.

Security & Stability: Enhances security and minimizes external impact from changes.

Ways to achieve Abstraction

1. Abstract class (0 to 100%)
2. Interface (100%)

Abstract Class

Declared with the abstract keyword is an abstract class.

May or may not contain abstract methods (methods without body).

Key Points:

- Abstract classes cannot be instantiated.
- They may contain both abstract (without implementation) and concrete (with implementation) methods.
- If a class contains at least one abstract method, it must be declared abstract.

Abstract Methods

Declared without implementation in an abstract class

Does not have a body

Implementation is provided by subclasses

```java
abstract class Animal {
    abstract void sound();  // Abstract method
    void sleep() {  // Concrete method
        System.out.println(x:"Sleeping");
    }
}
class Dog extends Animal {

    void sound() {
        System.out.println(x:"Bark");
    }
}
public class Example {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Dog obj = new Dog();
        obj.sound();
        obj.sleep();
    }
}
```

Characteristics of Abstract Classes

Objects cannot be created directly from an abstract class

May contain abstract and concrete methods (with body)

Can have constructors

Subclasses must provide implementations for all abstract methods

Abstract Class Example with Constructor

```java
abstract class Animal {
    String name;
    Animal(String name) {
        this.name = name;
    }
    abstract void sound();
}
class Dog extends Animal {
    Dog(String name) {
        super(name);
    }

    void sound() {
        System.out.println(name + " barks.");
    }
}
public class Example {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Dog dog = new Dog(name:"Buddy");
        dog.sound();  // Output: Buddy barks.
    }
}
```

Inheriting an Abstract Class

```java
abstract class Animal {
    abstract void sound();
}

class Dog extends Animal {

    void sound() {
        System.out.println(x:"Bark");
    }
}

public class Example {
    Run | Debug
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.sound();  // Output: Bark
    }
}
```

Abstraction and Encapsulation

Feature	**Abstraction**	**Inheritance**
Definition	Hides details, shows only essential functionality	Allows new classes to inherit behaviors from existing ones
Purpose	Simplifies complex systems	Promotes code reuse
Achieved Through	Abstract classes and interfaces	Extending existing classes
Focus	*What* an object does	*How* an object behaves
Example	Interface for sending messages	Dog class inherits from Animal

Interface

Reference type that contains abstract methods

Cannot create an instance of an interface.

Contains abstract methods, constants, default/static methods, and nested types.

All methods are implicitly abstract, except default and static methods

Includes only static and final fields

Class uses implements to implement an interface and must define all its methods unless the class is abstract

Properties of Interfaces

An interface is implicitly abstract

Methods are implicitly public and abstract

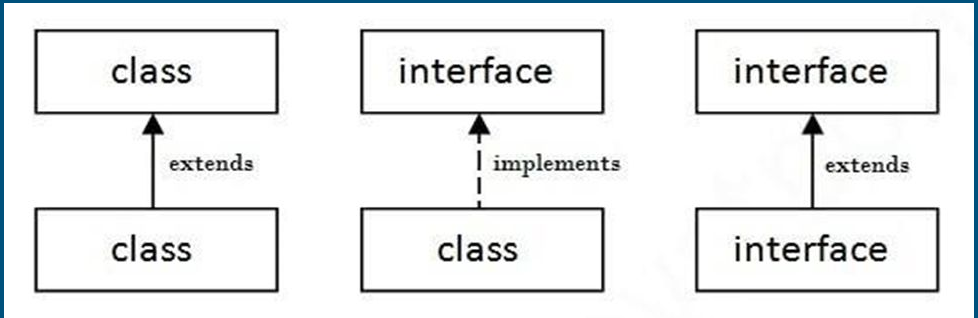
Cannot contain constructors

A class can implement multiple interfaces

Extending Interface

Can be extended using extends keyword

A class implements multiple interfaces, but can only extend one class



Tasks To Do

Week 11

Polymorphism

Simply means having many forms

Allows objects of different classes to be treated as objects of a common superclass

Enhances code flexibility, reusability, and maintainability by enabling to write more generic code that can work with various types of objects without needing to know their specific implementations

Methods for Polymorphism

Method Overloading: Same name but different parameters (Compile Time Polymorphism)

Method Overriding: Same name and parameters as parent class (Run Time Polymorphism)

Method Overloading

Allows you to define multiple methods in the same class with the same name but different parameter lists

Increases the readability of the program

Can be achieved by:

- By changing number of arguments
- By changing the data type

Changing the number of arguments

```java
class Sum {
    public int add(int a, int b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Sum obj = new Sum();
        System.out.println(obj.add(a:10, b:20));
        System.out.println(obj.add(a:10, b:20, c:30));
    }
}
```

Changing the data type

```
class Sum {
    public int add(int a, int b) {
        return a + b;
    }

//   changing datatype
    public double add(double a, double b) {
        return a + b;
    }
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Sum obj = new Sum();
        System.out.println(obj.add(a:10, b:20));
        System.out.println(obj.add(a:10.2, b:20.2));
    }
}
```

Method Overriding

Allows a subclass to provide a specific implementation for a method that is already defined in its superclass

Overridden method in the subclass should have the same name, return type, and parameters as the method in the superclass

@Override annotation is used to indicate that a method is intended to override a superclass method

Used to provide the specific implementation of a method that is already provided by its superclass

Used for runtime polymorphism

Rules for Overriding

The method must have the same name as in the parent class

The method must have the same parameters as the parent class.

There must be an IS-A relationship (inheritance).

Example

```java
class Animal {
    public void interact() {
        System.out.println(x:"Animal can interact");
    }
}
class Human extends Animal {
    @Override
    public void interact() {
        System.out.println(x:"Human can speak");
    }
}
public class Example {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Animal a = new Animal();
        a.interact();
        Human h = new Human();
        h.interact();

    }
}
```

Method Overloading and Overriding

Feature	Method Overloading	Method Overriding
Definition	Creating multiple methods with the same name but different parameters in the same class	Subclass provides a specific implementation of a method already defined in its superclass
Binding	Compile-time (static) binding	Run-time (dynamic) binding
Return Type	Can vary	Return type must be the same as or a subtype of the superclass method
Purpose	Used to perform the same operation in different ways, based on method arguments	Used to modify or extend the behavior of an inherited method.
Inheritance	Does not require inheritance	Requires inheritance
Number of Methods	Multiple methods with the same name but different parameter lists	One method in the superclass and one in the subclass.

Tasks To Do

Week 12

Exception

Unwanted event that disrupts normal program flow.

Causes program termination if not handled.

Java allows handling exceptions to improve user experience.

Common Causes:

- Division by zero.
- Accessing an array with an invalid index.
- Providing invalid user input.
- Attempting to open a non-existent file.

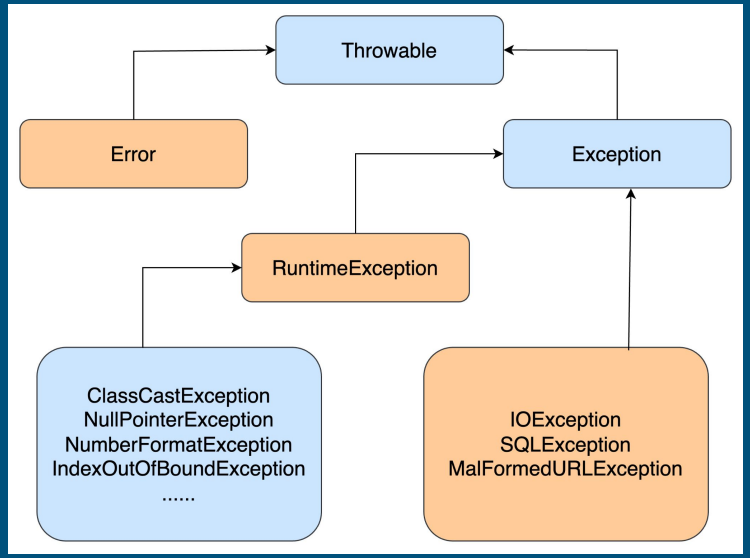
Errors and Exceptions

Error	**Exception**
Severe issues like hardware failures	Can be handled programmatically.
Should cause application crash	Allows corrective measures
Example: OutOfMemoryError.	Example: NullPointerException.

Hierarchy of Java Exception Classes

Exceptions are present in the java.lang.Throwable class

Exception and Error inherit the class, which are further inherited by other subclasses.



Types of Exceptions

Checked Exceptions

- Handled during compilation.
- Example: SQLException, IOException.

Unchecked Exceptions

- Occur at runtime.
- Example: NullPointerException, ArithmeticException.

Exception Handling

Crucial feature in Java that allows handling runtime errors

Ensures program continuity without abrupt termination.

Helps identify and fix runtime issues.

Improves user experience by providing clear error messages.

Java Exception Handling Keywords

try – Defines the block where an exception may occur.

catch – Handles the exception if it occurs.

finally – Executes regardless of exception occurrence.

throw – Used to manually throw an exception.

throws – Declares exceptions a method may throw.

Try-Catch Mechanism

```java
public class Example {
    Run | Debug
    public static void main(String[] args) {
        try {
            int num = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println(x:"Cannot divide by zero!");
        }

    }
}
```

Multiple Catch Blocks

A single try block can have multiple catch blocks

Specific exception handlers should be placed before generic ones

Finally Block

Used for cleanup code (closing files, releasing resources)

Executes regardless of whether an exception occurs

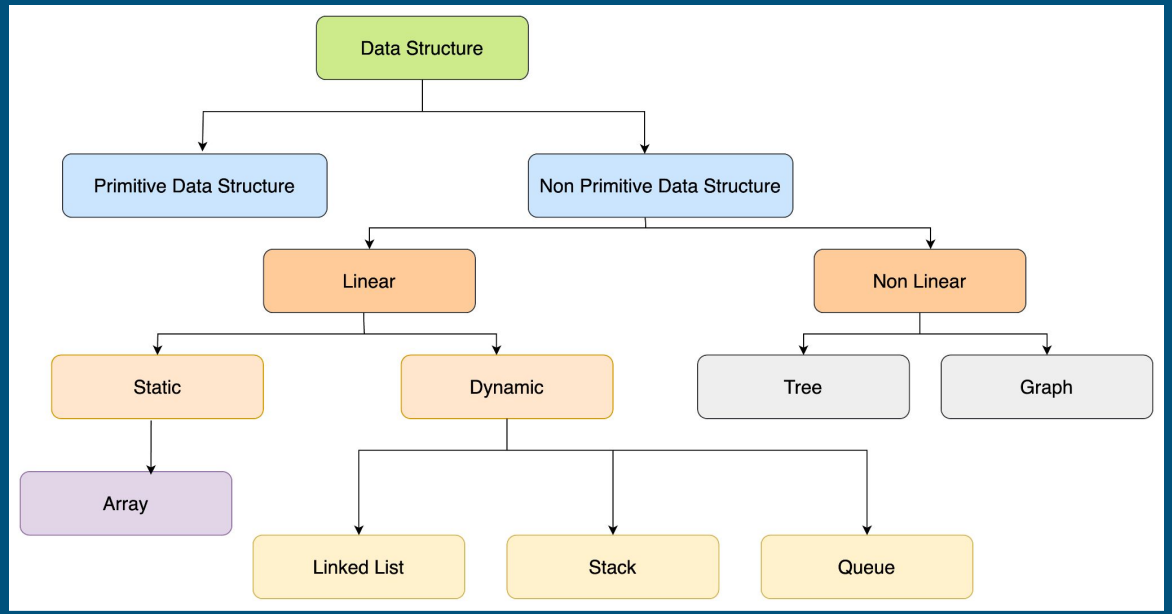
```
public class Example {
    Run | Debug
    public static void main(String[] args) {
        try {
            int data = 5 / 0;
        } catch (ArithmeticException e) {
            System.out.println(x:"Exception caught");
        } finally {
            System.out.println(x:"Finally block executed");
        }

    }
}
```

Week 13

Data Structures

Group of data elements which provides an efficient way of storing and organising data in the computer so that it can be used efficiently



Linear Data Structures

Array: collection of similar type of data items

Stack: linear list in which insertion and deletions are allowed only at one end, called top

Queue: linear list in which elements can be inserted only at one end called rear and deleted only at the other end called front

Stack

Follows Last In, First Out (LIFO) principle.

Think of a stack as a pile of plates.

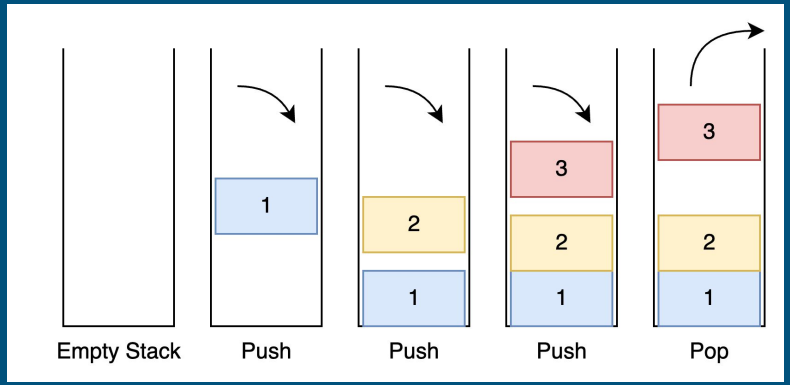
Operations:

Push: Add an element to the top.

Pop: Remove an element from the top.

Peek: Get the top element without removing it.

IsEmpty & IsFull: Check if stack is empty/full.



```java
class StackImplementation{
    int top;
    int size;
    int[] stack;
    StackImplementation(int size){
        this.size = size;
        stack = new int[size];
        top = -1;
    }
    void push(int data){
        if(isFull()){
            System.out.println(x:"Stack is full");
            return;
        }
        stack[++top] = data;
    }
    int pop(){
        if(isEmpty()){
            System.out.println(x:"Stack is empty");
            return -1;
        }
        return stack[top--];
    }
    int peek(){
        if(isEmpty()){
            System.out.println(x:"Stack is empty");
            return -1;
        }
        return stack[top];
    }
    boolean isEmpty(){
        return top == -1;
    }
    boolean isFull(){
        return top == size-1;
    }
}
```

```java
public class Example {
    Run | Debug
    public static void main(String[] args) {
        StackImplementation stack = new StackImplementation(size:5);
        stack.push(data:10);
        stack.push(data:20);
        stack.push(data:30);
        System.err.println("Stack is Full: "+stack.isFull());
        System.out.println(stack.pop());
        System.out.println(stack.peek());
        System.out.println("Stack is Empty: "+stack.isEmpty());
        System.out.println(stack.pop());
}
}
```

Output

```
Stack is Full: false
30
20
Stack is Empty: false
20
```

Applications of Stack

Expression Evaluation: Used in compilers for infix-to-postfix conversion.

Undo/Redo Functionality: In text editors and browsers.

Backtracking: Used in recursion and solving mazes.

Queue

Follows First In, First Out (FIFO) principle.

Think of a queue as a line at a ticket counter.

Operations:

Enqueue: Add an element at the rear.

Dequeue: Remove an element from the front.

Peek: Get the front element without removing it.

IsEmpty & IsFull: Check if queue is empty/full.

```java
class QueueImplementation{
    int front;
    int rear;
    int size;
    int[] queue;
    int capacity;
    QueueImplementation(int capacity){
        this.capacity = capacity;
        front = this.size = 0;
        rear = capacity - 1;
        queue = new int[this.capacity];
    }
    void enqueue(int item){
        if(isFull()){
            return;
        }
        rear = (rear + 1) % capacity;
        queue[rear] = item;
        size = size + 1;
    }
    int dequeue(){
        if(isEmpty()){
            return Integer.MIN_VALUE;
        }
        int item = queue[front];
        front = (front + 1) % capacity;
        size = size - 1;
        return item;
    }
    boolean isFull(){
        return size == capacity;
    }
    boolean isEmpty(){
        return size == 0;
    }
    int peek(){
        if(isEmpty()){
            return Integer.MIN_VALUE;
        }
        return queue[front];
    }
}

public class Example {
    Run | Debug
    public static void main(String[] args) {
        QueueImplementation queue = new QueueImplementation(capacity:1000);
        queue.enqueue(item:10);
        queue.enqueue(item:20);
        System.out.println(queue.dequeue() + " dequeued from queue\n");
        System.out.println("Front item is " + queue.peek());
        System.out.println("isFull: " + queue.isFull());
        System.out.println("isEmpty: " + queue.isEmpty());
}
}
```

Output

```
10 dequeued from queue

Front item is 20
isFull: false
isEmpty: false
```

Applications of Queue

CPU Scheduling & Disk Scheduling.

Handling Requests in Web Servers.

Call Center Systems & Real-Time Processing.

Stack and Queue

Feature	Stack	Queue
Insertion	Push (Top)	Enqueue (Rear)
Deletion	Pop (Top)	Dequeue (Front)
Achieved Order	Last In, First Out	First In, First Out
Examples	Undo/Redo, Function Calls	Task Scheduling, Printing Jobs

Tasks To Do

Week 14

Java Swing

Swing is a part of Java Foundation Classes (JFC).

Lightweight and platform-independent GUI toolkit.

Used for creating window-based applications.

Includes components like buttons, scroll bars, text fields, etc.

Container Class

A container holds other components.

Types of containers:

- Panel: Organizes components inside a window.
- Frame: A fully functional window with icons and a title.
- Dialog: A pop-up window, not as functional as a frame.

Java Swing

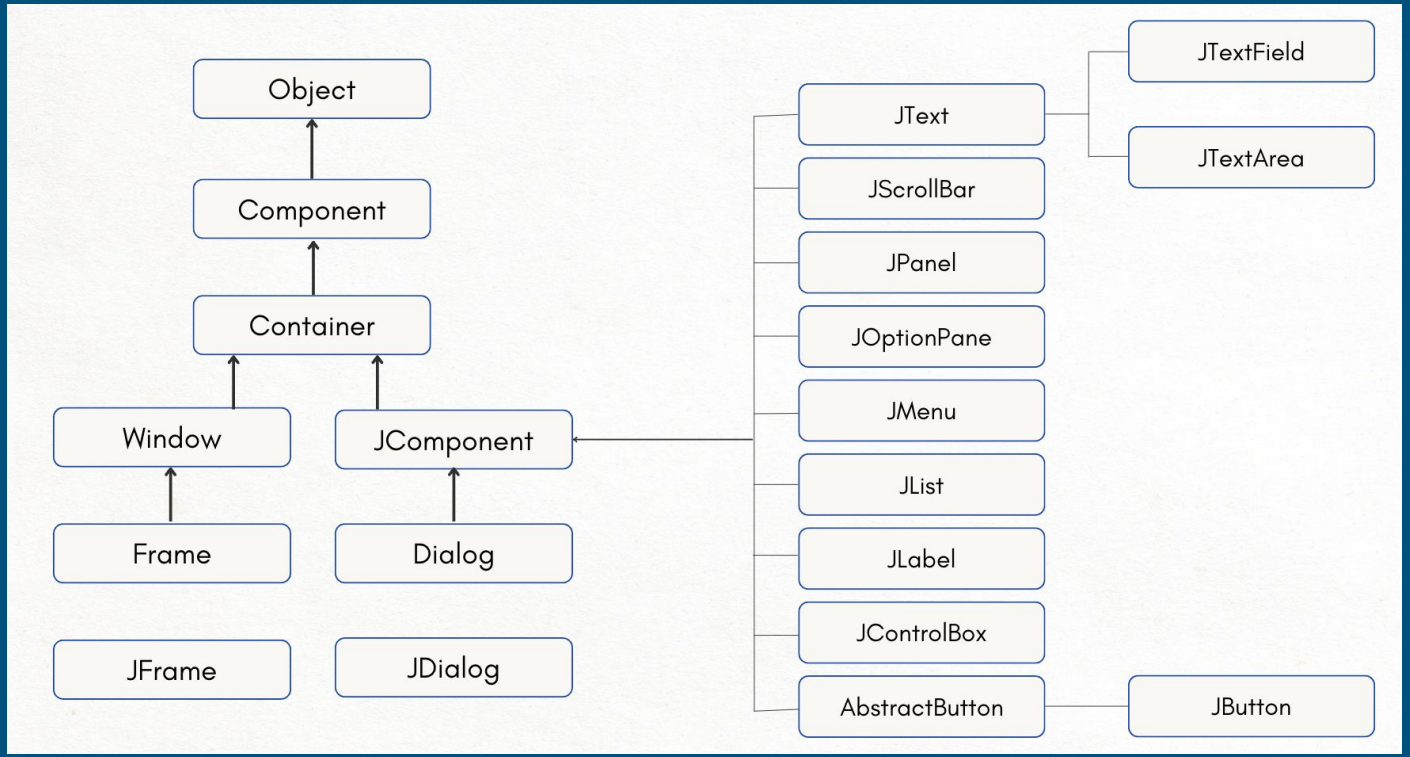
All Swing components inherit from JComponent.

Containers (JFrame, JDialog) hold components.

Components (JButton, JTextField, JLabel) are added to containers.

setLayout() method is used to override default layouts.

Java Swing Class Hierarchy



JFrame Class

Basic top-level window

Used to create main application window

```
JFrame frame = new JFrame("My App");
frame.setSize(400, 300);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
```

JPanel Class

Inherits JComponent.

Provides space to attach other components.

```
JFrame frame = new JFrame(title:"My App");
frame.setSize(width:400, height:300);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(b:true);

JPanel panel = new JPanel();
frame.add(panel);
```

JLabel Class

Used to display text in a container.

setText(String text): Sets the text displayed on the label.

getText(): Gets the current text of the label.

setIcon(Icon icon): Sets an icon to the label.

getIcon(): Gets the current icon of the label.

setHorizontalAlignment(int alignment): Sets the horizontal alignment (e.g., JLabel.LEFT, JLabel.CENTER).

setVerticalAlignment(int alignment): Sets the vertical alignment (e.g., JLabel.TOP, JLabel.CENTER).

setFont(Font font): Sets the font of the label text.

setForeground(Color c): Sets the text color.

setBackground(Color c): Sets the background color (requires setOpaque(true)).

setOpaque(boolean isOpaque): Sets whether the label is opaque (visible background).

setBorder(Border border): Sets a border for the label.

```java
JLabel label = new JLabel(text:"Initial Text");
label.setText(text:"Updated Text");                    // Change text
label.setIcon(new ImageIcon(filename:"icon.png"));      // Set an icon
label.setHorizontalAlignment(JLabel.CENTER);   // Align text to center
label.setFont(new Font(name:"Arial", Font.BOLD, size:16)); // Set font
label.setForeground(Color.RED);                // Set text color
label.setBackground(Color.YELLOW);             // Set background color
label.setOpaque(isOpaque:true);                          // Make label opaque
label.setBorder(BorderFactory.createLineBorder(Color.BLACK));  // Set border
panel.add(label);
```

JTextField Class

Used to edit single-line text.

setText(String text): Sets the text in the text field.

getText(): Gets the current text from the text field.

setEditable(boolean editable): Enables or disables editing.

setColumns(int columns): Sets the number of columns (width) of the text field.

setCaretPosition(int position): Sets the caret position in the text field.

setForeground(Color color): Sets the text color.

setBackground(Color color): Sets the background color.

selectAll(): Selects all text in the text field.

setFont(Font font): Sets the font for the text field.

```java
JTextField textField = new JTextField();
textField.setText(t:"Hello, World!");             // Set text
String text = textField.getText();               // Get text
textField.setEditable(b:false);                   // Disable editing
textField.setColumns(columns:20);                       // Set column width
textField.setCaretPosition(position:5);                  // Set caret position
textField.setForeground(Color.BLUE);             // Set text color
textField.setBackground(Color.LIGHT_GRAY);      // Set background color
textField.selectAll();                           // Select all text
textField.setFont(new Font(name:"Serif", Font.ITALIC, size:14));  // Set font
```

JTextArea Class

setText(String text): Sets the text in the text area.

getText(): Gets the current text from the text area.

setEditable(boolean editable): Enables or disables editing.

setBackground(Color color): Sets the background color.

setFont(Font font): Sets the font for the text area.

append(String text): Appends text at the end of the existing text.

```java
JTextArea textArea = new JTextArea();
textArea.setText("Initial Text");                 // Set text
String currentText = textArea.getText();          // Get text
textArea.setEditable(false);                      // Disable editing
textArea.setBackground(Color.WHITE);              // Set background color
textArea.setFont(new Font("Courier", Font.PLAIN, 14)); // Set font
textArea.append("Appended Text");                 // Append text
```

JPasswordField Class

setText(String text): Sets the text in the password field (shown as dots).

getPassword(): Gets the password as a char[] array.

setEchoChar(char echoChar): Sets the echo character for password input (default is *).

setEditable(boolean editable): Enables or disables editing.

setFont(Font font): Sets the font of the password field.

setForeground(Color color): Sets the text color.

setBackground(Color color): Sets the background color.

```java
JPasswordField passwordField = new JPasswordField();
passwordField.setText("password123");                // Set text (shown as dots)
char[] password = passwordField.getPassword();      // Get password as char array
passwordField.setEchoChar('*');                      // Set echo char to *
passwordField.setEditable(false);                    // Disable editing
passwordField.setFont(new Font("SansSerif", Font.PLAIN, 14)); // Set font
passwordField.setForeground(Color.RED);             // Set text color
passwordField.setBackground(Color.YELLOW);          // Set background color
```

JButton Class

setText(String text): Sets the text displayed on the button.

getText(): Gets the current text of the button.

setEnabled(boolean enabled): Enables or disables the button.

addActionListener(ActionListener listener): Adds an action listener to handle button clicks.

setBackground(Color color): Sets the background color.

setFont(Font font): Sets the font for the button.

setBorder(Border border): Sets the border of the button.

setToolTipText(String text): Sets a tooltip for the button.

```java
JButton button = new JButton(text:"Click Me!");
button.setText(text:"Submit");                    // Set text
button.setEnabled(b:true);                       // Enable button
button.addActionListener(e -> System.out.println("Button Clicked"));  // Add action listener
button.setBackground(Color.GREEN);               // Set background color
button.setFont(new Font("Arial", Font.BOLD, 16)); // Set font
button.setBorder(BorderFactory.createLineBorder(Color.BLACK));  // Set border
button.setToolTipText("Click to submit form");  // Set tooltip text
```

JCheckbox Class

setSelected(boolean selected): Sets whether the checkbox is selected.

isSelected(): Returns whether the checkbox is selected.

setText(String text): Sets the label next to the checkbox.

addItemListener(ItemListener listener): Adds an item listener to listen for selection changes.

setBackground(Color color): Sets the background color of the checkbox.

setFont(Font font): Sets the font for the checkbox.

```java
JCheckBox checkBox = new JCheckBox("Accept Terms");
checkBox.setSelected(true);                          // Select checkbox
boolean isChecked = checkBox.isSelected();           // Check if selected
checkBox.setText("Agree to Terms and Conditions"); // Set label text
checkBox.addItemListener(e -> System.out.println("Checkbox clicked")); // Add listener
checkBox.setBackground(Color.CYAN);                  // Set background color
checkBox.setFont(new Font("Serif", Font.PLAIN, 12)); // Set font
```

JRadioButton Class

setSelected(boolean selected): Sets whether the radio button is selected.

isSelected(): Returns whether the radio button is selected.

setText(String text): Sets the label next to the radio button.

addActionListener(ActionListener listener): Adds an action listener to the radio button.

setBackground(Color color): Sets the background color of the radio button.

setFont(Font font): Sets the font for the radio button.

```java
JRadioButton radioButton = new JRadioButton("Option 1");
radioButton.setSelected(true);                      // Select radio button
boolean isRadioSelected = radioButton.isSelected(); // Check if selected
radioButton.setText("Choose Option 1");             // Set label text
radioButton.addActionListener(e -> System.out.println("Radio button clicked")); // Add listener
radioButton.setBackground(Color.ORANGE);            // Set background color
radioButton.setFont(new Font("Arial", Font.BOLD, 14)); // Set font
```

JComboBox Class

setSelectedItem(Object anObject): Selects an item from the combo box.

getSelectedItem(): Gets the selected item from the combo box.

addItem(Object item): Adds an item to the combo box.

removeItem(Object item): Removes an item from the combo box.

setEditable(boolean editable): Sets whether the combo box is editable.

setBackground(Color color): Sets the background color.

setFont(Font font): Sets the font for the combo box.

```
JComboBox<String> comboBox = new JComboBox<>(new String[]{"Option 1", "Option 2"});
comboBox.setSelectedItem("Option 1");              // Set selected item
String selectedItem = (String) comboBox.getSelectedItem(); // Get selected item
comboBox.addItem("Option 3");                      // Add new item
comboBox.removeItem("Option 2");                   // Remove item
comboBox.setEditable(true);                        // Set combo box editable
comboBox.setBackground(Color.LIGHT_BLUE);          // Set background color
comboBox.setFont(new Font("Serif", Font.ITALIC, 12)); // Set font
```

Adding to Container

```
JFrame frame = new JFrame("Swing Example");  // Create a JFrame
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Close on exit
frame.setSize(400, 300);  // Set size of the frame

// Create components
JLabel label = new JLabel("Hello, Swing!");
JButton button = new JButton("Click Me");

// Add components to the frame
frame.add(label);
frame.add(button);

// Set the layout manager (optional)
frame.setLayout(new FlowLayout()); // Use a FlowLayout for simple component arrangement

frame.setVisible(true);  // Make the frame visible
```

Adding to Container with Custom Layout

```
JFrame frame = new JFrame("Custom Layout Example");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

// Set layout to null
frame.setLayout(null);

// Create components
JLabel label = new JLabel("Custom Layout Label");
JButton button = new JButton("Click Me");

// Set the bounds (x, y, width, height)
label.setBounds(50, 50, 150, 30);
button.setBounds(50, 100, 100, 30);

// Add components to the frame
frame.add(label);
frame.add(button);

frame.setSize(400, 300);  // Set the size of the frame
frame.setVisible(true);   // Make the frame visible
```

Layout Managers

Arranges components inside containers.

Automatically positions controls.

Different layout managers handle positioning differently.

Types of Layouts

- BorderLayout (Default for JFrame)
- FlowLayout
- GridLayout
- CardLayout
- GridBagLayout
- BoxLayout
- GroupLayout
- SpringLayout

BorderLayout in Java

Arranges components in five regions: NORTH, SOUTH, EAST, WEST, CENTER.

Default layout for JFrame.

Constructors:

BorderLayout() (No gaps)

BorderLayout(int hgap, int vgap) (Gaps between components)

FlowLayout

Arranges components in a line, one after another, in a flow

Default layout of an applet or panel

Fields of FlowLayout:

- LEFT: Aligns components to the left.
- RIGHT: Aligns components to the right.
- CENTER: Aligns components in the center.
- LEADING: Aligns components to the leading edge.
- TRAILING: Aligns components to the trailing edge.

FlowLayout Constructors

FlowLayout(): Default constructor with centered alignment and a 5 unit gap.

FlowLayout(int align): Creates with specified alignment and default 5 unit gap.

FlowLayout(int align, int hgap, int vgap): Creates with specified alignment and custom horizontal/vertical gaps.

GridLayout

Arranges components in a rectangular grid where each component occupies one cell

Constructors of GridLayout:

- GridLayout(): Creates a grid with one column per component.
- GridLayout(int rows, int columns): Creates a grid with the specified rows and columns.
- GridLayout(int rows, int columns, int hgap, int vgap): Creates a grid with the specified rows, columns, and custom gaps.

Event Handling

Mechanism that controls the event and decides what should happen if an event occurs

Event: Change in the state of an object

Types of Events:

Foreground Events: Direct interactions of user. For example; clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list, etc

Background Events: Require the interaction of end user.For example; Operating system interrupts, hardware or software failure, timer expires, an operation completion, etc